

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Bacharelado em Engenharia da Computação

Maicon Gomes Messias
Ramon Vinícius Silva Corrêa
Yuri Martins de Sa

Trabalho Prático - Compiladores
Compilador CF (Compila-Fofo)

Belo Horizonte

2025

Maicon Gomes Messias
Ramon Vinícius Silva Corrêa
Yuri Martins de Sa

Trabalho Prático - Compiladores
Compilador CF (Compila-Fofo)

Trabalho Prático - Compiladores – Curso de Engenharia da Computação da Pontifícia Universidade Católica de Minas Gerais.

Orientador: Felipe de Castro Belém

Belo Horizonte

2025

RESUMO

Este trabalho apresenta o desenvolvimento completo de um compilador para a linguagem Compila Fofo (CF), uma linguagem imperativa simples projetada com propósitos didáticos para a disciplina de Compiladores. O projeto abrange todas as etapas fundamentais da construção de um compilador: análise léxica, análise sintática, análise semântica e geração de código Assembly MIPS 32 bits. A linguagem CF foi especificada com tipos de dados básicos (Inteiro, Lógico, Caractere e String), estruturas de controle (Se/Senao, Enquanto, Para), operadores aritméticos, relacionais e lógicos, além de suporte a comentários. A gramática foi projetada para ser LL(1), facilitando a implementação de um parser descendente recursivo. O analisador léxico, fundamentado em expressões regulares e autômatos finitos determinísticos, realiza a tokenização do código-fonte com reconhecimento case-insensitive de palavras-chave. O analisador sintático valida a estrutura gramatical através da técnica de descida recursiva, com métodos encadeados para tratamento de precedência de operadores. A análise semântica, integrada ao parser, utiliza uma tabela de símbolos com suporte a escopos aninhados, verificando declaração de variáveis, compatibilidade de tipos em operações e atribuições. A geração de código Assembly MIPS traduz programas CF em código executável, contemplando operações aritméticas, estruturas de controle complexas e conversões implícitas de tipos. O compilador implementa otimizações como constant folding e concatenação inteligente de strings. Os testes realizados confirmaram a funcionalidade completa do sistema, com execução bem-sucedida dos códigos gerados no simulador MARS, validando a corretude de todas as fases de compilação.

Palavras-chave: Compiladores. Análise Léxica. Análise Sintática. Análise Semântica. Geração de Código. Assembly MIPS. Linguagens Formais. Autômatos Finitos. Gramáticas Livres de Contexto. Parser Descendente Recursivo.

LISTA DE FIGURAS

FIGURA 1 – Autômato Completo CF 8

FIGURA 2 – Resultado do .asm 30

SUMÁRIO

1 INTRODUÇÃO **1**

2 DESENVOLVIMENTO **3**

2.1 Definição da Linguagem CF 3

2.1.1 Tipos de Dados 3

2.1.2 Palavras-chave 4

2.1.3 Operadores e Precedência 4

2.1.4 Comentários 4

2.1.5 Estruturas Suportadas 5

2.2 Gramática em Alto Nível 6

2.2.1 Estrutura geral do programa 6

2.2.2 Comandos 7

2.2.3 Expressões 7

2.2.4 Construção dos Autômatos 7

2.3 Analisador Léxico 9

2.3.1 Interface e Funcionamento Geral 10

2.3.2 Estrutura Principal (Main) 10

2.3.3 Categorias de Tokens 12

2.3.4 Tratamento de Erros Léxicos 13

2.3.5 Implementação do Lexer 13

2.4 Analisador Sintático 14

2.5 Analisador Semântico 15

2.5.1 Tabela de Símbolos 16

2.5.2 Implementação do Parser 17

2.6 Geração de Código e Execução 19

2.7 Otimizações 19

2.8 Testes e Execução 20

2.8.1 Resultados Obtidos 30

3 CONCLUSÃO **31**
3.1 Resumo dos Principais Pontos Abordados 31
3.2 Conclusão 32

1 INTRODUÇÃO

Este documento apresenta a documentação técnica completa do **Compila Fofo (CF)**, um compilador desenvolvido como trabalho prático da disciplina de Compiladores. O projeto implementa, de forma integrada, todas as principais etapas do processo de compilação de uma linguagem imperativa simples:

- **Análise Léxica** – definição do conjunto de tokens da linguagem, construção dos autômatos finitos determinísticos (AFDs) e implementação do analisador léxico responsável por transformar o código-fonte em uma sequência de tokens válidos.
- **Análise Sintática** – definição e implementação da gramática da linguagem em um parser descendente recursivo (LL(1)), respeitando precedência e associatividade de operadores, e verificando se a sequência de tokens forma programas corretamente estruturados.
- **Análise Semântica** – verificação de tipos, uso correto de variáveis e regras semânticas da linguagem, por meio de uma tabela de símbolos com suporte a escopos aninhados, garantindo que programas sintaticamente corretos também façam sentido do ponto de vista semântico.
- **Geração de Código** – tradução do programa CF em código Assembly **MIPS 32 bits**, incluindo declaração de variáveis na seção `.data`, geração de instruções na seção `.text`, implementação de estruturas de controle (`Se`, `Enquanto`, `Para`) e suporte a operações aritméticas, lógicas e relacionais.
- **Execução e Otimizações** – execução do código gerado em simuladores como MARS e SPIM, e aplicação de otimizações simples, como CONSTANT FOLDING e concatenação inteligente de strings, além da proposta de otimizações futuras.

A linguagem CF foi projetada com fins didáticos, buscando ao mesmo tempo ser expressiva o bastante para demonstrar conceitos fundamentais de compiladores e simples o suficiente para permitir uma implementação completa dentro do escopo da disciplina. Ela oferece tipos básicos (`Inteiro`, `Logico`/`Lógico`, `Caractere` e um tipo

interno `STRING`), estruturas de controle usuais e um comando de saída (`Imprimir`) com suporte a concatenação de diferentes tipos.

2 DESENVOLVIMENTO

2.1 Definição da Linguagem CF

A linguagem **Compila Fofo (CF)** é uma linguagem imperativa simples, projetada para fins didáticos na disciplina de Compiladores. Um programa CF é composto por:

- declarações de variáveis;
- comandos de atribuição;
- estruturas de controle (`Se`, `Senao`, `Enquanto`, `Para`);
- comando de saída (`Imprimir`);
- expressões aritméticas, relacionais e lógicas.

Não há funções definidas pelo usuário; o programa é um bloco único ("escopo global"), possivelmente com escopos internos delimitados por chaves (por exemplo, dentro de `Se`, `Enquanto`, `Para`).

2.1.1 Tipos de Dados

A linguagem oferece os seguintes tipos:

- `Inteiro`: domínio dos números inteiros (por exemplo, 0, 1, -5). Suporta operações aritméticas (`+`, `-`, `*`, `/`, `%`, `**`) e relacionais.
- `Logico`/`Lógico`: domínio dos valores booleanos `Verdade` (1) e `Mentira` (0). Participam de operações lógicas e de comparações.
- `Caractere`: utilizado para armazenar texto (strings literais entre aspas), principalmente para saída.
- `STRING`: tipo interno do compilador, não exposto diretamente na sintaxe da linguagem, usado para representar literais de string e resultados de concatenação com `+`.

2.1.2 Palavras-chave

As principais palavras-chave (insensíveis a maiúsculas/minúsculas) são:

- Tipos: `Inteiro`, `Logico`/`Lógico`, `Caractere`;
- Controle de fluxo: `Se`, `Senao`/`Senão`, `Enquanto`, `Para`, `em`;
- Entrada/Saída: `Imprimir`;
- Literais lógicos: `Verdade`, `Mentira`.

Identificadores de variáveis são distintos de palavras-chave, mas a verificação de palavras-chave é feita de forma case-insensitive.

2.1.3 Operadores e Precedência

A linguagem suporta os seguintes operadores, com a precedência indicada do mais forte para o mais fraco:

1. Potenciação: `**` (associatividade À DIREITA);
2. Unário: `-` (negação numérica unária);
3. Multiplicativos: `*`, `/`, `%`;
4. Aditivos: `+`, `-` (incluindo concatenação de strings com `+`);
5. Relacionais: `=`, `<>`, `<`, `>`, `<=`, `>=`;
6. Lógico E: `&`;
7. Lógico OU: `^`.

2.1.4 Comentários

Dois tipos de comentários são suportados:

- Comentário de linha: `$ comentário até o fim da linha`;
- Comentário multilinha: `$$ comentário em várias linhas $$`.

O analisador léxico remove comentários antes de tokenizar o código-fonte.

2.1.5 Estruturas Suportadas

Exemplos de estruturas da linguagem CF:

Declaração de variáveis

```
Inteiro x;
Inteiro i <- 10;
Inteiro a <- 1, b <- 2, c;
Logico flag <- Verdade;
Caractere texto <- "Olá";
```

Atribuição

```
x <- 5;
y <- x + 10;
flag <- x > 5;
```

Estrutura condicional

```
Se (x > 0) {
    Imprimir("Positivo");
} Senao {
    Imprimir("Não positivo");
}

Se x = 5 {
    Imprimir("x é cinco");
}
```

Laço Enquanto

```
Enquanto (i < 10) {
    i <- i + 1;
    Imprimir("i = " + i);
}
```

Laço Para

```
Para i em (1, 10, 1) {
    Imprimir("Valor: " + i);
}

```

```
Para j em (10, 1, -1) {
    Imprimir("Decrescente: " + j);
}
```

Impressão com concatenação

```
Inteiro idade <- 25;
Imprimir("Idade: " + idade);
Imprimir("Resultado: " + (5 + 3));
Logico teste <- Verdade;
Imprimir("Teste: " + teste);
```

2.2 Gramática em Alto Nível

A gramática efetiva da linguagem é implementada no parser descendente recursivo, mas pode ser descrita em notação EBNF simplificada.

2.2.1 Estrutura geral do programa

$$\langle programa \rangle ::= \langle declaracaoOuComando \rangle^{*}\ \texttt{EOF}$$

$$\langle declaracaoOuComando \rangle ::= \langle declaracao \rangle \mid \langle comando \rangle$$

$$\langle declaracao \rangle ::= \langle tipo \rangle\ \langle listaDeclaracoes \rangle\ \texttt{;}$$

$$\langle tipo \rangle ::= \texttt{Inteiro} \mid \texttt{Logico} \mid \texttt{Caractere}$$

$$\langle listaDeclaracoes \rangle ::= \langle declaracaoVar \rangle\ \Big(\texttt{,}\ \langle declaracaoVar \rangle\Big)^{*}$$

$$\langle declaracaoVar \rangle ::= \texttt{IDENTIFICADOR}\ \Big[\texttt{<-}\ \langle expressao \rangle\Big]$$

2.2.2 Comandos

$$\langle comando \rangle ::= \langle comandoAtribuicao \rangle \mid \langle comandoSe \rangle \mid \langle comandoEnquanto \rangle \mid \langle comandoPara \rangle \mid \langle comandoImprimir \rangle \mid \langle bloco \rangle$$

$$\langle comandoAtribuicao \rangle ::= \texttt{IDENTIFICADOR <-} \langle expressao \rangle \texttt{ ;}$$

$$\langle comandoSe \rangle ::= \texttt{Se [(]} \langle expressao \rangle \texttt{[)]} \langle bloco \rangle \; [\texttt{Senao} \langle bloco \rangle]$$

$$\langle comandoEnquanto \rangle ::= \texttt{Enquanto [(]} \langle expressao \rangle \texttt{[)]} \langle bloco \rangle$$

$$\langle comandoPara \rangle ::= \texttt{Para IDENTIFICADOR em (} \langle expressao \rangle \texttt{ ,} \langle expressao \rangle \texttt{ ,} \langle expressao \rangle \texttt{)} \langle bloco \rangle$$

$$\langle comandoImprimir \rangle ::= \texttt{Imprimir (} \langle expressao \rangle \texttt{) ;}$$

$$\langle bloco \rangle ::= \texttt{\{} \langle declaracaoOuComando \rangle^{*} \texttt{\}}$$

2.2.3 Expressões

$$\langle expressao \rangle ::= \langle exprOu \rangle$$

$$\langle exprOu \rangle ::= \langle exprE \rangle \; (\texttt{\^{}} \; \langle exprE \rangle)^{*}$$

$$\langle exprE \rangle ::= \langle exprRel \rangle \; (\texttt{\&} \; \langle exprRel \rangle)^{*}$$

$$\langle exprRel \rangle ::= \langle exprAdd \rangle \; [\, (\texttt{=} \mid \texttt{<>} \mid \texttt{<} \mid \texttt{>} \mid \texttt{<=} \mid \texttt{>=}) \; \langle exprAdd \rangle \,]$$

$$\langle exprAdd \rangle ::= \langle exprMul \rangle \; ((\texttt{+} \mid \texttt{-}) \; \langle exprMul \rangle)^{*}$$

$$\langle exprMul \rangle ::= \langle exprPow \rangle \; ((\texttt{*} \mid \texttt{/} \mid \texttt{\%}) \; \langle exprPow \rangle)^{*}$$

$$\langle exprPow \rangle ::= \langle exprUn \rangle \; (\texttt{**} \; \langle exprUn \rangle)^{*}$$

$$\langle exprUn \rangle ::= \texttt{-} \; \langle exprUn \rangle \mid \langle primario \rangle$$

$$\langle primario \rangle ::= \texttt{IDENTIFICADOR} \mid \texttt{NUM_INTEIRO} \mid \texttt{STRING_LITERAL} \mid \texttt{Verdade} \mid \texttt{Mentira} \mid \texttt{(} \langle expressao \rangle \texttt{)}$$

2.2.4 Construção dos Autômatos

Na imagem a seguir vemos a construção do autômato completo do nosso projeto:

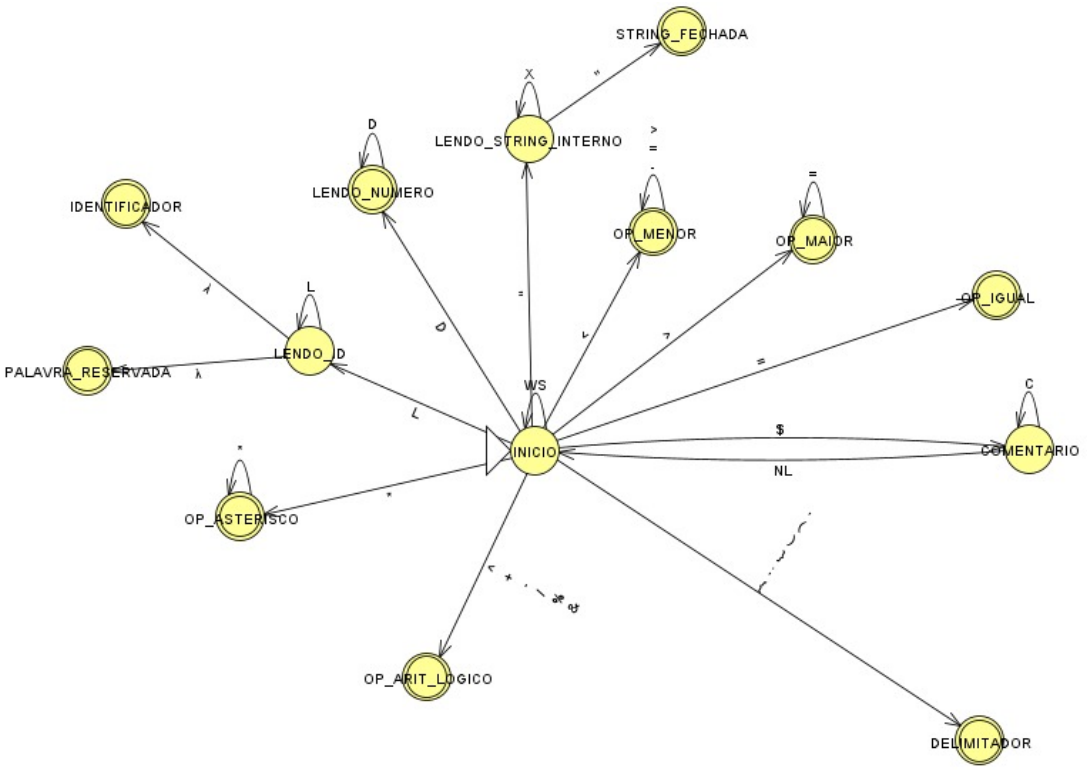


Figura 1 – Autômato Completo CF

O autômato abaixo representa, de forma unificada, o analisador léxico da linguagem.

- Conjunto de estados:

$$Q = \{\texttt{INICIO}, \texttt{LENDO_ID}, \texttt{PALAVRA_RESERVADA}, \texttt{IDENTIFICADOR}, \texttt{LENDO_NUMERO}, \texttt{LENDO_STRING_INTERNO}, \texttt{STRING_FECHADA}, \texttt{OP_MENOR}, \texttt{OP_MAIOR}, \texttt{OP_IGUAL}, \texttt{OP_ASTERISCO}, \texttt{OP_ARIT_LOGICO}, \texttt{COMENTARIO}, \texttt{DELIMITADOR}\}.$$

- Alfabeto (agrupado em classes):

$$\Sigma = \{L, D, ., <, >, =, *, \text{op_arit/log}, \$, WS, NL, \text{delim}, X, C\}$$

onde L = letra ou _, D = dígito, WS = espaço/branco, NL = quebra de linha, delim = delimitadores como `( ) , ; { }`, X = caractere interno de string e C = qualquer caractere de comentário.

- Estado inicial: `INICIO`.

- Estados de aceitação (geram tokens):

  $F = \{$`PALAVRA_RESERVADA`, `IDENTIFICADOR`, `LENDO_NUMERO`, `STRING_FECHADA`,
  `OP_MENOR`, `OP_MAIOR`, `OP_IGUAL`, `OP_ASTERISCO`,
  `OP_ARIT_LOGICO`, `DELIMITADOR`$\}$.

  O estado `COMENTARIO` é não-aceitador (o texto é descartado).

- Transições principais (esboço):
  - De `INICIO`:
    - com L vai para `LENDO_ID`;
    - com D vai para `LENDO_NUMERO`;
    - com " vai para `LENDO_STRING_INTERNO`;
    - com $<$ vai para `OP_MENOR`;
    - com $>$ vai para `OP_MAIOR`;
    - com $=$ vai para `OP_IGUAL`;
    - com $*$ vai para `OP_ASTERISCO`;
    - com operador aritm./lógico simples vai para `OP_ARIT_LOGICO`;
    - com \$ vai para `COMENTARIO`;
    - com delimitadores vai para `DELIMITADOR`;
    - com WS ou NL permanece em `INICIO`.
  - Em `LENDO_ID`, laço com L (e dígitos); ao final, transição λ para `PALAVRA_RESERVADA` ou `IDENTIFICADOR`, conforme a tabela de palavras-chave.
  - Em `LENDO_NUMERO`, laço com D; ao ler outro símbolo, o token número é reconhecido.
  - Em `LENDO_STRING_INTERNO`, laço com X e, ao ler ", vai para `STRING_FECHADA`.
  - Nos estados `OP_MENOR`, `OP_MAIOR`, `OP_IGUAL` e `OP_ASTERISCO`, o segundo caractere (por exemplo `=`, `>`, `*`) decide se o token é um operador simples ou composto (`<=`, `>=`, `<>`, `**`, etc.).
  - Em `COMENTARIO`, laço com C até ler NL, quando retorna a `INICIO` sem gerar token.

2.3 Analisador Léxico

O analisador léxico da linguagem CF é implementado na classe `lexer/Lexer.java`. Seu papel é ler o código-fonte de entrada como uma sequência de caracteres e produzir uma sequência de TOKENS bem tipados, que serão consumidos pelo analisador sintático.

2.3.1 *Interface e Funcionamento Geral*

O lexer recebe como entrada o conteúdo de um arquivo `.cf` e percorre o texto caractere a caractere, aplicando, de forma conceitual, os autômatos finitos determinísticos (AFDs). A cada reconhecimento bem-sucedido, é criado um objeto `Token`, cujo tipo pertence ao enum `Token.Tipo` definido em `lexer/Token.java`.

Na implementação atual, o lexer utiliza expressões regulares (REGEX) que refletem as mesmas linguagens reconhecidas pelos AFDs projetados. Há uma ordem cuidadosa na tentativa de reconhecimento:

- remoção de comentários (linha e multilinha) antes da tokenização, usando regex equivalentes aos AFDs;
- reconhecimento prioritário de operadores de dois caracteres (`**`, `>=`, `<=`, `<>`, `<-`);
- em seguida, reconhecimento de operadores e símbolos de um caractere (`+`, `-`, `*`, `/`, `%`, parênteses, chaves, vírgulas, ponto e vírgula etc.);
- reconhecimento de literais numéricos, literais de string e identificadores/palavras-chave.

Palavras-chave são reconhecidas de forma CASE-INSENSITIVE: o lexema é normalizado e comparado com uma lista de palavras reservadas. Identificadores são armazenados em minúsculas para facilitar comparações posteriores.

2.3.2 *Estrutura Principal (Main)*

A classe `Main` atua como o orquestrador do compilador. Ela é responsável por ler o arquivo de código fonte fornecido, instanciar o analisador léxico (`Lexer`), exibir os tokens identificados (para fins de depuração) e, em seguida, passar a lista de tokens para o analisador sintático (`Parser`).

O fluxo de execução foi dividido em etapas visuais no console para facilitar o acompanhamento do processo de compilação.

```
1  public class Main {
2      public static void main(String[] args) {
3          try {
4              // Definição do arquivo de entrada e saída
5              Path arquivo = Path.of("inputfiles/exemplo.cf");
6              String codigo = Files.readString(arquivo);
7
```

```
8               System.out.println("========================================");
9               System.out.println("       COMPILADOR - COMPILA FOFO       ");
10              System.out.println("========================================");
11              System.out.println("CODIGO FONTE: \n" + codigo);
12              System.out.println("----------------------------------------");
13
14              // ETAPA 1: Análise Léxica
15              System.out.println("ETAPA 1 - ANALISE LEXICA");
16              Lexer lexer = new Lexer(codigo);
17              List<Token> tokens = lexer.analisar();
18
19              // Exibe os tokens encontrados
20              tokens.forEach(t -> System.out.println(t));
21
22              System.out.println("----------------------------------------");
23
24              // ETAPA 2 e 3: Análise Sintática/Semântica e Geração de Código
25              System.out.println("ETAPA 2 - ANALISE SINTATICA / SEMANTICA");
26              Parser parser = new Parser(tokens);
27              parser.analisar(); // Inicia o processo de parsing
28
29              // Se não houver erros, gera o arquivo de saída
30              System.out.println("----------------------------------------");
31              System.out.println("ETAPA 3 - GERACAO DE CODIGO (MIPS)");
32              String codigoMIPS = parser.getCodigoGeradoMIPS();
33              Files.writeString(Path.of("outputfiles/saida.asm"), codigoMIPS);
34
35              System.out.println("\nCompilacao concluida com sucesso!");
36              System.out.println("Codigo salvo em: outputfiles/saida.asm");
37
38          } catch (IOException e) {
39              System.err.println("Erro ao ler/escrever arquivos: " + e.getMessage());
40          } catch (RuntimeException e) {
41              System.err.println("Erro de compilacao: " + e.getMessage());
42          } catch (Exception e) {
43              System.err.println("Erro inesperado: ");
44              e.printStackTrace();
45          }
46      }
47  }
```

Código 1 – Classe Main: Ponto de entrada do compilador

2.3.3 Categorias de Tokens

Cada lexema reconhecido é classificado em uma categoria de token, como por exemplo:

- tipos (`INTEIRO`, `LOGICO`, `CARACTERE`);
- identificadores (`IDENTIFICADOR`);
- literais inteiros (`NUM_INTEIRO`);
- literais de string (`STRING_LITERAL`);
- operadores aritméticos, lógicos e relacionais;
- delimitadores (parênteses, chaves, vírgulas, ponto e vírgula);
- palavras-chave de controle (`SE`, `SENAO`, `ENQUANTO`, `PARA`, `EM`) e de entrada/saída (`IMPRIMIR`);
- literais lógicos (`VERDADE`, `MENTIRA`).

Essas categorias estão definidas no enum `Token.Tipo` e são utilizadas pelo parser para tomar decisões sintáticas.

```
1  package lexer;
2
3  public enum Tipo {
4      INTEIRO, LOGICO, CARACTERE,
5      ENQUANTO, SE, SENAO, PARA,
6      IMPRIMIR, VERDADE, MENTIRA,
7      NUMERO, STRING, IDENTIFICADOR,
8      OPERADOR_ARIT, OPERADOR_LOGICO, ATRIBUICAO,
9      ABRE_CHAVE, FECHA_CHAVE,
10     ABRE_PAREN, FECHA_PAREN,
11     VIRGULA, PONTO_VIRGULA, FIM
12 }
```

Código 2 – Enumeração dos Tipos de Token

A classe `Token` armazena o tipo e o valor (lexema) encontrado.

```
1  package lexer;
2
3  public class Token {
```

```
4       private Tipo tipo;
5       private String valor;
6
7       public Token(Tipo tipo, String valor) {
8           this.tipo = tipo;
9           this.valor = valor;
10      }
11
12      public Tipo getTipo() { return tipo; }
13      public String getValor() { return valor; }
14
15      @Override
16      public String toString() {
17          return "<" + tipo + ", " + valor + ">";
18      }
19  }
```

Código 3 – Classe Token

2.3.4 Tratamento de Erros Léxicos

Erros léxicos ocorrem quando uma sequência de caracteres não pertence a nenhuma das linguagens reconhecidas pelos AFDs definidos (por exemplo, um caractere inválido, uma string não terminada ou um número seguido de letra de forma incorreta). Nesses casos:

- o lexer identifica o ponto de falha (posição no código);
- emite uma mensagem de erro informando o caractere ou lexema inesperado;
- interrompe o processo de análise, impedindo que etapas posteriores (sintática e semântica) sejam executadas sobre uma sequência de tokens inválida.

Dessa forma, o analisador léxico atua como primeira linha de defesa, garantindo que somente tokens válidos cheguem ao parser.

2.3.5 Implementação do Lexer

O `Lexer` percorre a string de entrada utilizando a classe `Matcher` do Java. A expressão regular utilizada inclui flags para ignorar maiúsculas/minúsculas (`?iu`) em palavras-chave.

```
public class Lexer {
    private String codigo;
    private List<Token> tokens;

    // Regex completa extraída do projeto
    private static final String REGEX =
        "(?iu)(Inteiro|Logico|Caractere|Enquanto|Senao|Se|Para|Imprimir|Verdade|
    Mentira)" + // Palavras-chave
        "|(\\*\\*|>=|<=|<>|<-)" +       // Operadores compostos
        "|(\\d+)" +                     // Números
        "|(\\p{L}+)" +                  // Identificadores (letras)
        "|([+\\-/%*])" +                // Operadores aritméticos simples
        "|(=|>|<|&|\\^)" +              // Operadores relacionais/lógicos
        "|([{};,()])" +                 // Delimitadores
        "|(\"[^\"]*\")";                // Strings literais

    public Lexer(String codigo) {
        this.codigo = codigo;
        this.tokens = new ArrayList<>();
    }

    public List<Token> analisar() {
        // Remove comentários de bloco ( ... ) e linha (
    ...)StringcodigoLimpo = codigo.replaceAll("(?s)\\.*?\\", ).replaceAll("(?m).*
```

Código 4 – Classe Lexer e Expressões Regulares

2.4 Analisador Sintático

O analisador sintático, implementado em `parser/Parser.java`, é um parser descendente recursivo (LL(1)) que consome os tokens gerados pelo lexer e verifica se a sequência pertence à gramática definida na Seção **??**.

A precedência de operadores é implementada por meio de métodos encadeados, em que cada nível corresponde a um conjunto de operadores:

```
expressao -> exprOr -> exprAnd -> exprRel ->
exprAdd -> exprMul -> exprPow -> exprUn -> primario
```

Cada não terminal da gramática é implementado como um método no parser, como por exemplo `parseExpressao`, `parseComandoSe`, `parseComandoEnquanto`, `parseComandoPara`, `parseBloco`, entre outros.

Erros sintáticos são detectados quando um token esperado não é encontrado (falha de correspondência em `match`) ou quando a ordem dos tokens viola a gramática (por exemplo, ausência de ponto e vírgula, chaves ou parênteses obrigatórios). Nesses casos, o compilador emite mensagens de erro claras e interrompe a compilação.

2.5 Analisador Semântico

A análise semântica é responsável por garantir que o programa CF, além de sintaticamente correto, faça sentido em termos de tipos e uso de variáveis. Essa etapa é implementada em conjunto com o parser e utiliza duas estruturas principais:

- `TipoDado` (em `Tipo_de_dados/TipoDado.java`): enumeração que representa os tipos internos (INTEIRO, LOGICO, CARACTERE, STRING etc.).
- `TabelaDeSimbolos` (em `TabelaDeSimbolos/TabelaDeSimbolos.java`): estrutura de dados que gerencia os símbolos (variáveis) e seus tipos, com suporte a escopos aninhados por meio de uma pilha de mapas.

As principais regras semânticas verificadas incluem:

- variáveis devem ser declaradas antes do uso;
- não é permitido uso de variáveis não declaradas;
- operações aritméticas exigem operandos do tipo INTEIRO;
- operações lógicas exigem operandos do tipo LOGICO;
- operações relacionais operam sobre inteiros e retornam LOGICO;
- atribuições verificam compatibilidade entre o tipo da expressão e o tipo da variável;
- condições de estruturas de controle (`Se`, `Enquanto`, `Para`) devem ser do tipo LOGICO;
- concatenação com `+` aceita STRING, INTEIRO e LOGICO, com conversão implícita para STRING quando necessário.

Erros semânticos (como uso de variável não declarada, incompatibilidade de tipos em operações ou atribuições inválidas) são reportados ao usuário e impedem a geração de código.

2.5.1 Tabela de Símbolos

A tabela de símbolos foi implementada como uma pilha de mapas, permitindo o suporte a escopos aninhados (variáveis locais e globais).

```
1
2  public class TabelaDeSimbolos {
3      private List<Map<String, TipoDado>> escopos;
4
5      public TabelaDeSimbolos() {
6          this.escopos = new ArrayList<>();
7          abrirEscopo(); // Escopo global
8      }
9
10     public void abrirEscopo() {
11         escopos.add(new HashMap<>());
12     }
13
14     public void fecharEscopo() {
15         if (!escopos.isEmpty()) {
16             escopos.remove(escopos.size() - 1);
17         }
18     }
19
20     public boolean declarar(String nome, TipoDado tipo) {
21         Map<String, TipoDado> escopoAtual = escopos.get(escopos.size() - 1);
22         if (escopoAtual.containsKey(nome)) {
23             return false; // Variável já declarada no escopo atual
24         }
25         escopoAtual.put(nome, tipo);
26         return true;
27     }
28
29     public TipoDado buscar(String nome) {
30         // Busca do escopo mais interno para o mais externo
31         for (int i = escopos.size() - 1; i >= 0; i--) {
32             if (escopos.get(i).containsKey(nome)) {
33                 return escopos.get(i).get(nome);
34             }
35         }
36         return null; // Não encontrada
37     }
38 }
```

Código 5 – Gerenciamento de Escopo na Tabela de Símbolos

2.5.2 Implementação do Parser

O parser implementa a técnica de descida recursiva. Abaixo estão listados os métodos principais que tratam comandos como `Se/Senao` e `Declaracao`, incluindo a verificação de tipos e chamadas para geração de código.

```
1
2  public class Parser {
3      private List<Token> tokens;
4      private int atual = 0;
5      private TabelaDeSimbolos ts;
6      // Variáveis para geração de código omitidas (CodeGenMIPS)
7
8      public Parser(List<Token> tokens) {
9          this.tokens = tokens;
10         this.ts = new TabelaDeSimbolos();
11     }
12
13     public void analisar() {
14         while (!isAtEnd()) {
15             comando();
16         }
17         System.out.println("ANALISE SINTATICA REALIZADA COM SUCESSO.");
18         System.out.println("ANALISE SEMANTICA REALIZADA COM SUCESSO.");
19     }
20
21     private void comando() {
22         Token t = tokens.get(atual);
23         if (t.getTipo() == Tipo.INTEIRO || t.getTipo() == Tipo.LOGICO) {
24             declaracao();
25         } else if (t.getTipo() == Tipo.SE) {
26             cmdSeSenao();
27         } else if (t.getTipo() == Tipo.ENQUANTO) {
28             cmdEnquanto();
29         } else if (t.getTipo() == Tipo.IMPRIMIR) {
30             cmdImprimir();
31         } else if (t.getTipo() == Tipo.IDENTIFICADOR) {
32             cmdAtribuicao();
33         } else {
34             erroSintatico("Comando invalido ou inesperado: " + t.getValor());
35         }
36     }
37
38     private void cmdSeSenao() {
39         consumir(Tipo.SE, "Esperado 'Se'");
40         consumir(Tipo.ABRE_PAREN, "Esperado '(' apos Se");
```

```
41
42          // Avalia expressão lógica
43          Res condicao = expressao();
44          verificarLogico(condicao);
45
46          consumir(Tipo.FECHA_PAREN, "Esperado ')' apos condicao");
47          consumir(Tipo.ABRE_CHAVE, "Esperado '{'");
48
49          ts.abrirEscopo();
50          while (check(Tipo.FECHA_CHAVE) == false) {
51              comando();
52          }
53          ts.fecharEscopo();
54          consumir(Tipo.FECHA_CHAVE, "Esperado '}'");
55
56          if (match(Tipo.SENAO)) {
57              consumir(Tipo.ABRE_CHAVE, "Esperado '{' apos Senao");
58              ts.abrirEscopo();
59              while (check(Tipo.FECHA_CHAVE) == false) {
60                  comando();
61              }
62              ts.fecharEscopo();
63              consumir(Tipo.FECHA_CHAVE, "Esperado '}' apos bloco Senao");
64          }
65      }
66
67      private void declaracao() {
68          Token tipoTok = tokens.get(atual);
69          avancar(); // consome tipo
70
71          Token id = consumir(Tipo.IDENTIFICADOR, "Esperado identificador na declaracao"
    );
72
73          // Verifica semântica: Declaração duplicada
74          if (!ts.declarar(id.getValor(), converterTipo(tipoTok.getTipo()))) {
75              erroSemantico("Variavel ja declarada neste escopo: " + id.getValor());
76          }
77
78          consumir(Tipo.PONTO_VIRGULA, "Esperado ';' ao final da declaracao");
79      }
80
81      // Métodos auxiliares
82      private Token consumir(Tipo tipo, String msg) {
83          if (check(tipo)) return avancar();
84          throw new RuntimeException("Erro Sintatico: " + msg);
85      }
86
```

```
87      private boolean check(Tipo tipo) {
88          if (isAtEnd()) return false;
89          return tokens.get(atual).getTipo() == tipo;
90      }
91  }
```

Código 6 – Implementação parcial do Parser e análise de comandos

2.6 Geração de Código e Execução

A etapa de geração de código traduz o programa CF em código Assembly MIPS 32 bits, implementada na classe `codegen/CodeGenMIPS.java`. O fluxo é:

- recebimento da estrutura sintática anotada com tipos e informações da tabela de símbolos;
- geração da seção `.data`, contendo variáveis globais, strings literais e buffers auxiliares;
- geração da seção `.text`, contendo o código executável com rótulo de entrada `main`;
- tradução de cada comando CF em uma sequência de instruções MIPS correspondentes.

Operações aritméticas (`+`, `-`, `*`, `/`, `%`) são mapeadas para instruções MIPS apropriadas, enquanto a potenciação (`**`) é implementada com um laço de multiplicações sucessivas. Operadores relacionais e lógicos são traduzidos em comparações e saltos condicionais, gerando valores 0 ou 1 em registradores temporários.

Estruturas de controle (`Se`/`Senao`, `Enquanto`, `Para`) são convertidas em rótulos e instruções de desvio (`beq`, `bne`, `j`, etc.), formando o fluxo de controle correspondente no Assembly.

A saída final é escrita no arquivo `outputfiles/saida.asm`, que pode ser executado em simuladores como MARS ou SPIM.

2.7 Otimizações

O compilador CF implementa algumas otimizações simples:

- **Constant Folding**: expressões compostas apenas por literais numéricos (por exemplo, `10 + 5 * 2 - 8 / 2`) são avaliadas em tempo de compilação, reduzindo o número de instruções geradas.

- **Concatenação inteligente**: tratamento especial da operação + em contexto de strings, evitando conversões desnecessárias e unificando o tratamento de STRING, INTEIRO e LOGICO.
- **Geração de rótulos únicos**: uso sistemático de rótulos distintos para regiões de código (if/else, laços), evitando colisões e simplificando o fluxo de execução.

2.8 Testes e Execução

Para realizar os testes utilizamos diversos códigos de entrada na linguagem CF. A seguir temos um exemplo de código que foi executado no compilador (Teste 6: operadores lógicos AND (&) e OR (^)):

```
$ TESTE 6 - Logico, AND (&) e OR (^)

Logico a <- Verdade;
Logico b <- Mentira;
Logico c <- Mentira;

Logico r <- a & b;
Logico s <- a ^ b;
Logico t <- a ^ b ^ c;

Imprimir("TESTE 6:");
Imprimir("a = " + a);
Imprimir("b = " + b);
Imprimir("c = " + c);
Imprimir("a & b = " + r);
Imprimir("a ^ b = " + s);
Imprimir("a ^ b ^ c = " + t);
```

Código 7 – Código de teste CF – Teste 6

Com esse código de entrada temos o seguinte código *.asm* de saída:

```
.data
int_buffer: .space 32

a: .word 0
b: .word 0
c: .word 0
r: .word 0
t0: .word 0
s: .word 0
```

```
10 t1: .word 0
11 t: .word 0
12 t2: .word 0
13 t3: .word 0
14 str_0: .asciiz "TESTE 6:"
15 str_1: .asciiz "a = "
16 str_concat_2: .space 512
17 str_3: .asciiz "b = "
18 str_concat_4: .space 512
19 str_5: .asciiz "c = "
20 str_concat_6: .space 512
21 str_7: .asciiz "a & b = "
22 str_concat_8: .space 512
23 str_9: .asciiz "a ^ b = "
24 str_concat_10: .space 512
25 str_11: .asciiz "a ^ b ^ c = "
26 str_concat_12: .space 512
27
28 .text
29 .globl main
30 main:
31   li $t0, 1
32   sw $t0, a
33   li $t0, 0
34   sw $t0, b
35   li $t0, 0
36   sw $t0, c
37   lw $t1, a
38   lw $t2, b
39   and $t3, $t1, $t2
40   sw $t3, t0
41   lw $t0, t0
42   sw $t0, r
43   lw $t1, a
44   lw $t2, b
45   or  $t3, $t1, $t2
46   sw $t3, t1
47   lw $t0, t1
48   sw $t0, s
49   lw $t1, a
50   lw $t2, b
51   or  $t3, $t1, $t2
52   sw $t3, t2
53   lw $t1, t2
54   lw $t2, c
55   or  $t3, $t1, $t2
56   sw $t3, t3
```

```
  lw $t0, t3
  sw $t0, t
  la $a0, str_0
  li $v0, 4
  syscall
  # intToString(a) -> int_buffer
  lw $t0, a
  la $t1, int_buffer
  li $t2, 0
  beq $t0, $zero, M3
M0:
  li $t3, 10
  div $t0, $t3
  mfhi $t4
  mflo $t0
  addi $t4, $t4, 48
  sb $t4, 0($t1)
  addi $t1, $t1, 1
  addi $t2, $t2, 1
  bne $t0, $zero, M0
  sb $zero, 0($t1)
  la $t1, int_buffer
  move $t4, $t1
  addu $t5, $t1, $t2
  addi $t5, $t5, -1
M1:
  blt $t5, $t4, M2
  lb $t6, 0($t4)
  lb $t7, 0($t5)
  sb $t7, 0($t4)
  sb $t6, 0($t5)
  addi $t4, $t4, 1
  addi $t5, $t5, -1
  j M1
M2:
  j M4
M3:
  li $t4, 48
  sb $t4, 0($t1)
  addi $t1, $t1, 1
  sb $zero, 0($t1)
M4:
  # concatStrings(str_1, int_buffer)
  la $t0, str_concat_2
  la $t1, str_1
M5:
  lb $t2, 0($t1)
```

```
104   beq $t2, $zero, M6
105   sb $t2, 0($t0)
106   addi $t0, $t0, 1
107   addi $t1, $t1, 1
108   j M5
109 M6:
110   la $t1, int_buffer
111 M7:
112   lb $t2, 0($t1)
113   beq $t2, $zero, M8
114   sb $t2, 0($t0)
115   addi $t0, $t0, 1
116   addi $t1, $t1, 1
117   j M7
118 M8:
119   sb $zero, 0($t0)
120   la $a0, str_concat_2
121   li $v0, 4
122   syscall
123   # intToString(b) -> int_buffer
124   lw $t0, b
125   la $t1, int_buffer
126   li $t2, 0
127   beq $t0, $zero, M12
128 M9:
129   li $t3, 10
130   div $t0, $t3
131   mfhi $t4
132   mflo $t0
133   addi $t4, $t4, 48
134   sb $t4, 0($t1)
135   addi $t1, $t1, 1
136   addi $t2, $t2, 1
137   bne $t0, $zero, M9
138   sb $zero, 0($t1)
139   la $t1, int_buffer
140   move $t4, $t1
141   addu $t5, $t1, $t2
142   addi $t5, $t5, -1
143 M10:
144   blt $t5, $t4, M11
145   lb $t6, 0($t4)
146   lb $t7, 0($t5)
147   sb $t7, 0($t4)
148   sb $t6, 0($t5)
149   addi $t4, $t4, 1
150   addi $t5, $t5, -1
```

```
  j M10
M11:
  j M13
M12:
  li $t4, 48
  sb $t4, 0($t1)
  addi $t1, $t1, 1
  sb $zero, 0($t1)
M13:
  # concatStrings(str_3, int_buffer)
  la $t0, str_concat_4
  la $t1, str_3
M14:
  lb $t2, 0($t1)
  beq $t2, $zero, M15
  sb $t2, 0($t0)
  addi $t0, $t0, 1
  addi $t1, $t1, 1
  j M14
M15:
  la $t1, int_buffer
M16:
  lb $t2, 0($t1)
  beq $t2, $zero, M17
  sb $t2, 0($t0)
  addi $t0, $t0, 1
  addi $t1, $t1, 1
  j M16
M17:
  sb $zero, 0($t0)
  la $a0, str_concat_4
  li $v0, 4
  syscall
  # intToString(c) -> int_buffer
  lw $t0, c
  la $t1, int_buffer
  li $t2, 0
  beq $t0, $zero, M21
M18:
  li $t3, 10
  div $t0, $t3
  mfhi $t4
  mflo $t0
  addi $t4, $t4, 48
  sb $t4, 0($t1)
  addi $t1, $t1, 1
  addi $t2, $t2, 1
```

```
  bne $t0, $zero, M18
  sb $zero, 0($t1)
  la $t1, int_buffer
  move $t4, $t1
  addu $t5, $t1, $t2
  addi $t5, $t5, -1
M19:
  blt $t5, $t4, M20
  lb $t6, 0($t4)
  lb $t7, 0($t5)
  sb $t7, 0($t4)
  sb $t6, 0($t5)
  addi $t4, $t4, 1
  addi $t5, $t5, -1
  j M19
M20:
  j M22
M21:
  li $t4, 48
  sb $t4, 0($t1)
  addi $t1, $t1, 1
  sb $zero, 0($t1)
M22:
  # concatStrings(str_5, int_buffer)
  la $t0, str_concat_6
  la $t1, str_5
M23:
  lb $t2, 0($t1)
  beq $t2, $zero, M24
  sb $t2, 0($t0)
  addi $t0, $t0, 1
  addi $t1, $t1, 1
  j M23
M24:
  la $t1, int_buffer
M25:
  lb $t2, 0($t1)
  beq $t2, $zero, M26
  sb $t2, 0($t0)
  addi $t0, $t0, 1
  addi $t1, $t1, 1
  j M25
M26:
  sb $zero, 0($t0)
  la $a0, str_concat_6
  li $v0, 4
  syscall
```

```
  # intToString(r) -> int_buffer
  lw $t0, r
  la $t1, int_buffer
  li $t2, 0
  beq $t0, $zero, M30
M27:
  li $t3, 10
  div $t0, $t3
  mfhi $t4
  mflo $t0
  addi $t4, $t4, 48
  sb $t4, 0($t1)
  addi $t1, $t1, 1
  addi $t2, $t2, 1
  bne $t0, $zero, M27
  sb $zero, 0($t1)
  la $t1, int_buffer
  move $t4, $t1
  addu $t5, $t1, $t2
  addi $t5, $t5, -1
M28:
  blt $t5, $t4, M29
  lb $t6, 0($t4)
  lb $t7, 0($t5)
  sb $t7, 0($t4)
  sb $t6, 0($t5)
  addi $t4, $t4, 1
  addi $t5, $t5, -1
  j M28
M29:
  j M31
M30:
  li $t4, 48
  sb $t4, 0($t1)
  addi $t1, $t1, 1
  sb $zero, 0($t1)
M31:
  # concatStrings(str_7, int_buffer)
  la $t0, str_concat_8
  la $t1, str_7
M32:
  lb $t2, 0($t1)
  beq $t2, $zero, M33
  sb $t2, 0($t0)
  addi $t0, $t0, 1
  addi $t1, $t1, 1
  j M32
```

```
M33:
  la $t1, int_buffer
M34:
  lb $t2, 0($t1)
  beq $t2, $zero, M35
  sb $t2, 0($t0)
  addi $t0, $t0, 1
  addi $t1, $t1, 1
  j M34
M35:
  sb $zero, 0($t0)
  la $a0, str_concat_8
  li $v0, 4
  syscall
  # intToString(s) -> int_buffer
  lw $t0, s
  la $t1, int_buffer
  li $t2, 0
  beq $t0, $zero, M39
M36:
  li $t3, 10
  div $t0, $t3
  mfhi $t4
  mflo $t0
  addi $t4, $t4, 48
  sb $t4, 0($t1)
  addi $t1, $t1, 1
  addi $t2, $t2, 1
  bne $t0, $zero, M36
  sb $zero, 0($t1)
  la $t1, int_buffer
  move $t4, $t1
  addu $t5, $t1, $t2
  addi $t5, $t5, -1
M37:
  blt $t5, $t4, M38
  lb $t6, 0($t4)
  lb $t7, 0($t5)
  sb $t7, 0($t4)
  sb $t6, 0($t5)
  addi $t4, $t4, 1
  addi $t5, $t5, -1
  j M37
M38:
  j M40
M39:
  li $t4, 48
```

```
  sb $t4, 0($t1)
  addi $t1, $t1, 1
  sb $zero, 0($t1)
M40:
  # concatStrings(str_9, int_buffer)
  la $t0, str_concat_10
  la $t1, str_9
M41:
  lb $t2, 0($t1)
  beq $t2, $zero, M42
  sb $t2, 0($t0)
  addi $t0, $t0, 1
  addi $t1, $t1, 1
  j M41
M42:
  la $t1, int_buffer
M43:
  lb $t2, 0($t1)
  beq $t2, $zero, M44
  sb $t2, 0($t0)
  addi $t0, $t0, 1
  addi $t1, $t1, 1
  j M43
M44:
  sb $zero, 0($t0)
  la $a0, str_concat_10
  li $v0, 4
  syscall
  # intToString(t) -> int_buffer
  lw $t0, t
  la $t1, int_buffer
  li $t2, 0
  beq $t0, $zero, M48
M45:
  li $t3, 10
  div $t0, $t3
  mfhi $t4
  mflo $t0
  addi $t4, $t4, 48
  sb $t4, 0($t1)
  addi $t1, $t1, 1
  addi $t2, $t2, 1
  bne $t0, $zero, M45
  sb $zero, 0($t1)
  la $t1, int_buffer
  move $t4, $t1
  addu $t5, $t1, $t2
```

```
386   addi $t5, $t5, -1
387 M46:
388   blt $t5, $t4, M47
389   lb $t6, 0($t4)
390   lb $t7, 0($t5)
391   sb $t7, 0($t4)
392   sb $t6, 0($t5)
393   addi $t4, $t4, 1
394   addi $t5, $t5, -1
395   j M46
396 M47:
397   j M49
398 M48:
399   li $t4, 48
400   sb $t4, 0($t1)
401   addi $t1, $t1, 1
402   sb $zero, 0($t1)
403 M49:
404   # concatStrings(str_11, int_buffer)
405   la $t0, str_concat_12
406   la $t1, str_11
407 M50:
408   lb $t2, 0($t1)
409   beq $t2, $zero, M51
410   sb $t2, 0($t0)
411   addi $t0, $t0, 1
412   addi $t1, $t1, 1
413   j M50
414 M51:
415   la $t1, int_buffer
416 M52:
417   lb $t2, 0($t1)
418   beq $t2, $zero, M53
419   sb $t2, 0($t0)
420   addi $t0, $t0, 1
421   addi $t1, $t1, 1
422   j M52
423 M53:
424   sb $zero, 0($t0)
425   la $a0, str_concat_12
426   li $v0, 4
427   syscall
```

Código 8 – Código ASM de saída

2.8.1 Resultados Obtidos

Ao executar o código no simulador assembly MARS, obtivemos o seguinte resultado:

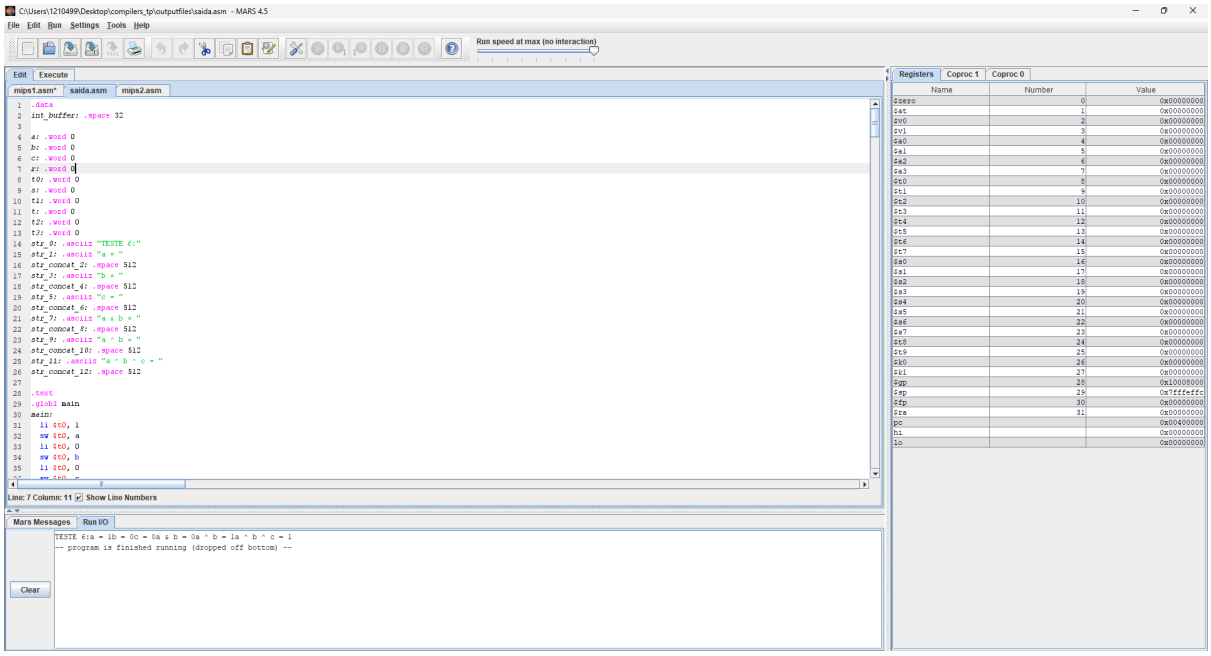


Figura 2 – Resultado do .asm

Como vimos na figura 2, Temos o seguinte resultado na saída:

```
TESTE 6:
a = 1
b = 0
c = 0
a & b = 0
a ^ b = 1
a ^ b ^ c = 1
-- program is finished running (dropped off bottom) --
```

Código 9 – Saída do programa de teste

O resultado foi exatamente o esperado executando as operações lógicas explicítas no Código 7. O código completo mostrado durante a documentação está disponibilizado no GitHub através do link: `github.com/ramonVSCorrea/compilers_tp.git`.

3 CONCLUSÃO

3.1 Resumo dos Principais Pontos Abordados

Este trabalho apresentou o desenvolvimento completo de um compilador para a linguagem **Compila Fofo (CF)**, uma linguagem imperativa simples projetada com fins didáticos para a disciplina de Compiladores. O projeto abrangeu todas as etapas fundamentais da construção de um compilador, desde a análise léxica até a geração de código executável em Assembly MIPS 32 bits.

A linguagem CF foi especificada com um conjunto reduzido mas expressivo de funcionalidades, incluindo tipos de dados básicos (Inteiro, Lógico, Caractere e String), estruturas de controle (Se/Senao, Enquanto, Para), operadores aritméticos, relacionais e lógicos, além de suporte a comentários de linha e multilinha. A gramática foi cuidadosamente desenhada para ser LL(1), facilitando a implementação de um parser descendente recursivo.

O **analisador léxico**, implementado através de expressões regulares e fundamentado em autômatos finitos determinísticos, demonstrou eficácia na tokenização do código-fonte, realizando o reconhecimento case-insensitive de palavras-chave e tratando adequadamente operadores compostos (`**`, `<=`, `>=`, `<>`, `<-`). A remoção de comentários foi integrada de forma elegante ao processo de análise, garantindo que apenas tokens válidos fossem encaminhados às fases subsequentes.

O **analisador sintático**, desenvolvido como um parser descendente recursivo, validou com sucesso a estrutura gramatical dos programas CF. A implementação de métodos encadeados para tratamento de precedência de operadores (`expressao` → `exprOu` → `exprE` → `exprRel` → `exprAdd` → `exprMul` → `exprPow` → `exprUn` → `primario`) mostrou-se efetiva e alinhada com os princípios teóricos de gramáticas livres de contexto.

A **análise semântica**, integrada ao parser através da tabela de símbolos com suporte a escopos aninhados, garantiu a corretude dos programas quanto ao uso de variáveis e compatibilidade de tipos. As verificações semânticas implementadas – como detecção de variáveis não declaradas, validação de tipos em operações aritméticas, lógicas e relacionais, e verificação de compatibilidade em atribuições – proporcionaram robustez ao

compilador, prevenindo a geração de código incorreto.

A **geração de código Assembly MIPS**, etapa final do processo de compilação, traduziu com êxito os programas CF em código executável. A implementação contemplou desde operações aritméticas básicas até estruturas de controle complexas, incluindo laços aninhados e conversões implícitas de tipos para concatenação de strings. As otimizações aplicadas, como CONSTANT FOLDING e concatenação inteligente, demonstraram preocupação não apenas com a corretude, mas também com a eficiência do código gerado.

Os **testes realizados**, exemplificados pelo Teste 6 que validou operações lógicas AND e OR, confirmaram a funcionalidade completa do compilador. A execução bem-sucedida dos códigos Assembly gerados no simulador MARS atestou a corretude de todas as fases de compilação, desde a tokenização até a geração de código.

3.2 Conclusão

O projeto alcançou plenamente seus objetivos ao entregar um compilador funcional e didaticamente valioso para a linguagem Compila Fofo. A experiência prática de implementar cada fase do processo de compilação – léxica, sintática, semântica e geração de código – proporcionou uma compreensão profunda dos conceitos teóricos estudados na disciplina de Compiladores.

A modularização do código em classes bem definidas (`Lexer`, `Parser`, `TabelaDeSimbolos`, `CodeGenMIPS`) facilitou o desenvolvimento incremental e a manutenção do projeto, evidenciando boas práticas de engenharia de software. A escolha de implementar o analisador sintático através da técnica de descida recursiva mostrou-se acertada, resultando em código legível e de fácil depuração.

As ferramentas e conceitos fundamentais da teoria de linguagens formais e autômatos foram aplicados com sucesso: expressões regulares modelaram os padrões léxicos, autômatos finitos fundamentaram o reconhecimento de tokens, e gramáticas livres de contexto definiram a estrutura sintática da linguagem. A integração entre análise sintática e semântica demonstrou como diferentes níveis de abstração cooperam para validar a corretude de um programa.

A geração de código Assembly MIPS representou o desafio de traduzir construções de alto nível em instruções de máquina, exigindo conhecimento profundo tanto da linguagem fonte quanto da arquitetura alvo. A implementação de rotinas auxiliares para conversão de inteiros em strings e concatenação de strings em Assembly ilustrou a complexidade envolvida na tradução de operações aparentemente simples.

Como trabalhos futuros, o compilador CF poderia ser estendido com funcionali-

dades adicionais, tais como:

- suporte a funções e procedimentos definidos pelo usuário, com passagem de parâmetros e retorno de valores;
- implementação de arrays e estruturas de dados mais complexas;
- otimizações mais avançadas, como eliminação de código morto, propagação de constantes e alocação inteligente de registradores;
- geração de código para outras arquiteturas (x86, ARM, RISC-V);
- desenvolvimento de um ambiente integrado de desenvolvimento (IDE) com destacamento de sintaxe, depuração e execução integrada;
- análise estática avançada para detecção de erros em tempo de compilação (uso de variáveis não inicializadas, divisão por zero, etc.).

Em síntese, o compilador Compila Fofo demonstra que, mesmo com recursos limitados, é possível construir uma ferramenta funcional e educativa que abrange todos os aspectos fundamentais de um compilador real. Este projeto consolida o aprendizado teórico através da prática, evidenciando que os princípios estudados em sala de aula são diretamente aplicáveis na construção de sistemas de software complexos e relevantes.